

Null Models and Sampling-Based Evaluation Expose the Real Value of Reinforcement Learning for LLVM Pass Ordering

Dimitrije Pešić

[Afiliacija]

Belgrade, Serbia

dimitrije.pesicc@gmail.com

Abstract—Reinforcement learning (RL) for compiler pass ordering is commonly evaluated by comparing a deterministic policy rollout against a fixed optimization level such as `-O3`. We show that both choices can reverse conclusions. We introduce two null models defined in the same curated 36-pass action space that the learned policies act in—a single random episode and best-of- N random search—and evaluate policies by sampling rather than by their argmax mode. Across 380 programs from seven independent sources, random search in the curated space alone beats `-Oz` on every real-code suite (1.6–24.5%), while a graph-neural policy trained on only four small programs beats the budget-matched random null on 24 of 24 suite–seed combinations — significant within every suite (one-sided Wilcoxon, largest $p=0.025$) — reaches exhaustive greedy-search quality at 73% of its compilation budget, and produces `.text` sections 10.6% smaller than `-Oz` with pass sequences that apply faster than the `-Oz` pipeline itself. An identically trained flat-feature policy achieves neither. Finally, RL gradients alone fail to train the graph encoder; distilling hand-crafted features into it before RL removes this failure. Measurement choices, not representation choices, dominate the reported conclusions in this domain.

Index Terms—compiler optimization, LLVM, phase ordering, reinforcement learning, graph neural networks, evaluation methodology

I. INTRODUCTION

The order in which a compiler applies its optimization passes changes the quality of the generated code, and choosing a good order per program — the *phase-ordering problem* — is NP-hard. Fixed pipelines such as LLVM’s `-O3` and `-Oz` apply one hand-tuned sequence to every program, which makes program-specific orderings an attractive target for machine learning, and a line of work has applied deep reinforcement learning to the problem [1], [2], [3], [4].

Reported results in this area, however, rest on two evaluation conventions that are rarely questioned. First, learned policies are compared against fixed optimization levels — often `-O3`, a *speed*-oriented pipeline, even when the objective is code *size* — rather than against a null model that measures how much of the gain is available without any learning. Second, policies are evaluated by a single deterministic (argmax) rollout, which measures the mode of the policy distribution rather than the distribution itself.

This paper asks what remains of an RL pass-ordering system once both conventions are replaced. We train proximal policy

optimization (PPO) [5] agents with two state representations — the flat 56-dimensional Autophase feature vector [1] and a custom GraphSAGE [6] encoder over control- and data-flow graphs parsed from LLVM IR — under a controlled protocol in which the representation is the only variable, and we evaluate them on 380 programs drawn from seven independent sources against null models defined in the same action space and with the same step budget.

Our contributions are threefold. (1) We show that a curated 36-pass action space is itself the dominant effect: best-of-50 random search inside it matches greedy search and beats `-Oz` on every real-code suite, a fact that any evaluation in this domain should control for. (2) We show that the evaluation mode reverses the ranking of representations: under argmax rollouts the two agents are indistinguishable, while under best-of- k sampling the graph-based policy beats the paired random null on every one of 24 suite–seed combinations and reaches exhaustive greedy-search quality at 73% of its compilation budget — a property the flat-feature policy does not have. (3) We show that RL gradients alone fail to train the graph encoder, and that distilling Autophase features into the encoder before RL removes the failure, improving deterministic-rollout generalization by 9–10%. All results are reproducible from a public repository with pinned dependencies.¹

II. RELATED WORK

AutoPhase [1] introduced deep RL over a flat feature vector for phase ordering; CompilerGym [2] standardized the environment we build on. ProGraML [7] showed that graph representations of programs outperform flat features on supervised compiler tasks, and MLGO [8] deployed learned policies inside production LLVM for inlining and register allocation. POSET-RL [3] and the coreset approach of Liang et al. [4] reduce the action space before learning; the latter is the closest work to ours, selecting from a coreset of 50 pass sequences with a GNN and reporting an average 4.7% improvement over `-Oz`. Our study is complementary: rather than proposing a new agent, it measures *where such gains come from*, using null models defined inside the same reduced space — a control absent from prior evaluations — and shows that the decoding

¹<https://github.com/dimitrijepesic/compiler-opt>

mode (argmax vs. sampling) changes conclusions. Mammadli et al. [9] observed that random search is a strong competitor to RL in this domain; we formalize that observation into an evaluation protocol and replicate it across seven suites. Large language models have also been applied to this problem [10], at a very different computational budget.

III. EXPERIMENTAL PROTOCOL

Environment and metric. We use CompilerGym 0.2.5 (llvm-ic-v0, LLVM 10) [2]. The optimization objective is LLVM IR instruction count (IC); Section IV-D validates IC against actual binary section sizes. $-O3/-Oz$ baselines are the environment’s real optimization-level observations, not approximated pass lists.

Action space. Profiling all 124 available passes on the training programs yields 36 passes that each improve at least one program; these cover 100% of the observed single-pass improvement. All agents, null models, and searches operate in this space with 45-step episodes.

Agents and training. Both agents use PPO (clip 0.2, $\gamma=0.99$, GAE $\lambda=0.95$) with truncation-aware advantage bootstrapping, KL-based early stopping of update epochs (target 0.02), and a linearly decaying entropy coefficient. The Autophase agent observes a $\log(1+x)$ -transformed 56-dimensional feature vector; the GNN agent parses the IR into a graph (one-hot opcode plus four scalar features per instruction; separate control-flow and data-flow edge types) and encodes it with a three-layer edge-type-aware GraphSAGE network. Both train on the *same* four small cBench programs (<3000 instructions), for the same 100 000 environment steps, with three seeds each; checkpoints are selected on a three-program validation subset. Apart from the joint optimizer that the shared trainable encoder requires (encoder learning rate 10^{-4}), the representation is the only variable.

Null models. Two null models are measured in the same 36-pass space: (a) a single random 45-step episode (the null for one policy rollout) and (b) the best of N random episodes (the null for search). When a policy is evaluated by best-of- k sampling, its null is the best of the *same number* k of stored random episodes on the same program — paired by construction.

Evaluation. Policies are evaluated in two modes: a single deterministic argmax rollout, and best-of- k stochastic rollouts sampled from the policy distribution (GNN rollouts sample with the encoder’s dropout active, which adds exploration noise). The evaluation battery spans seven sources: cBench (val+test, 9), MiBench [11] (40), CHStone [12] (12), NPB (122), BLAS (50), a 97-program POJ-104 sample, and 50 csmith-generated [13] synthetic programs — 371 battery programs plus cBench, none seen in training. Battery policy evaluations use programs up to 6000 instructions (347 of 371) to bound graph-extraction cost.

Encoder pretraining. In one arm, the GNN encoder is first trained to regress the Autophase vector from the program graph (2430 states collected by random episodes on the training programs) and then fine-tuned with RL.

TABLE I
TOTAL IR INSTRUCTION COUNT PER SUITE: REAL OPTIMIZATION LEVELS VS. BEST-OF-50 RANDOM EPISODES IN THE CURATED 36-PASS SPACE.

Suite (n)	O0	-O3	-Oz	Rnd-50	ΔOz
MiBench (40)	12 270	6 317	4 841	4765	+1.6%
BLAS (50)	40 745	39 141	37 838	36757	+2.9%
cBench (9)	232 442	163 834	115 987	111 006	+4.3%
CHStone (12)	19 857	14 182	9 834	9097	+7.5%
POJ-104 (97)	21 099	11 083	8 334	7622	+8.5%
NPB (122)	130 327	70 659	59 542	44 981	+24.5%
csmith (50)	352 963	82 950	57 625	60 626	-5.2%

IV. RESULTS

A. The curated action space, not learning, is the main effect

Table I compares real optimization levels against best-of-50 random episodes in the 36-pass space. Random search beats $-Oz$ on all six real-code suites, by 1.6% (MiBench) up to 24.5% (NPB); on NPB even a *single* random episode beats $-Oz$ by 11% on average. Two boundaries are equally consistent: $-O3$ is the wrong yardstick for size (on small programs it *increases* IC above $-O0$), and on synthetic csmith programs the full $-Oz$ pipeline retains a 5.2% advantage — the curated space was distilled from real code and transfers across real code only. Any learned-policy result in this domain should therefore be read relative to these nulls, not to optimization levels alone.

B. Sampling reverses the ranking of representations

Under argmax rollouts, both trained policies transfer poorly to programs one to two orders of magnitude larger than their training set — the GNN policy’s deterministic mode degenerates (e.g., 79 686 total IC on NPB against an $-Oz$ of 53 085) — and the two representations are statistically indistinguishable. Sampling the same checkpoints changes the picture completely. Table II reports best-of-8 sampled rollouts against the paired best-of-8 random null: the GNN policy beats its null on *every* suite-seed combination — 24 of 24 across the seven sources. Because the three seeds within a suite are compared against the same stored null episodes, we test at the level of the eight independent suite/split units: the GNN policy wins all eight under every seed (exact sign test $p=3.9\times 10^{-3}$); one-sided Wilcoxon tests against the per-program nulls are significant within every suite (largest $p=0.025$ on MiBench, down to $p<10^{-5}$ on NPB and POJ-104); and re-pairing each seed with a disjoint slice of the stored episodes, which restores independence across all 24 pairs, leaves the count at 24 of 24. The identically trained Autophase policy wins 15 of 24 pairs, two of the eight units, and is significant on four of the seven suites. The margin over the null is modest (1–2%) but perfectly systematic, and it holds on the synthetic csmith programs where pure random search *loses* to $-Oz$: there the sampled GNN policy is 16% below $-Oz$. A policy trained on four toy programs thus carries transferable ordering knowledge

TABLE II
BEST-OF-8 SAMPLED ROLLOUTS (MEDIAN SEED) VS. THE PAIRED
BEST-OF-8 RANDOM NULL AND -Oz. POLICIES WERE TRAINED ON FOUR
SMALL CBENCH PROGRAMS ONLY.

Suite (n)	Null	-Oz	AP	GNN
MiBench (40)	4 828	4 841	4 783	4 782
CHStone (12)	9 241	9 834	9 133	9 120
BLAS (50)	37 099	37 838	36 998	36 847
csmith (28)	17 555	20 226	17 179	17 064
NPB (120)	40 301	53 085	40 822	39 909
POJ-104 (97)	7 853	8 334	7 917	7 730

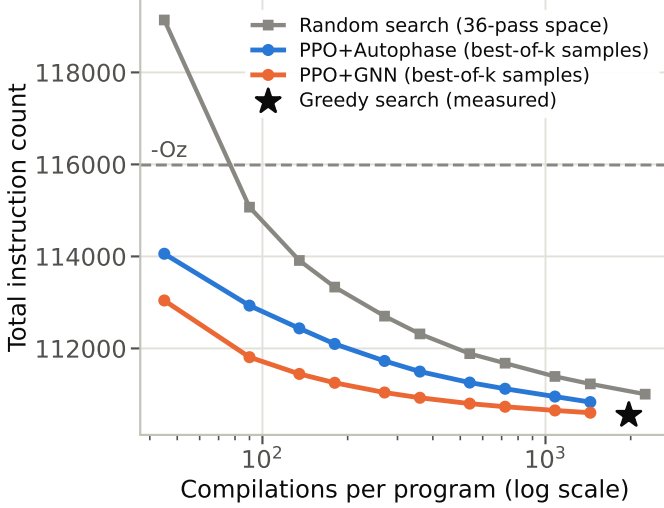


Fig. 1. Solution quality vs. compilation budget on cBench val+test: best-of- k policy sampling ($k=1..32$) against the expected best-of- k of random search in the same space and measured greedy search.

to seven foreign program sources — but only sampling can see it.

Fig. 1 places these results on a quality-vs.-budget front on cBench. At $k=8$ (360 compilations per program) the sampled GNN policy is within 0.3% of exhaustive greedy search, which costs about 1 970 compilations per program; at $k=32$ (73% of greedy’s budget) it reaches greedy quality, nominally surpassing it in two of three seeds (110 494 and 110 527 vs. 110 550), while all three seeds beat the paired null. Within the measured budgets random search never catches the sampler on cBench; on the battery it does so only at $k=32$, and first on NPB — precisely the suite where random search is strongest. The learned sampler, in other words, buys down the cost of search: it dominates the front at the budgets where search is expensive.

C. Pretraining unlocks the encoder

All three from-scratch GNN seeds converge to the same validation plateau (689), the value a nearly untrained policy already reaches: RL gradients alone do not teach the encoder useful features. Distilling Autophase into the encoder before RL (a one-hour CPU job) removes the failure in two of two seeds (Table III), and improves full-split deterministic-rollout

TABLE III
EFFECT OF AUTOPHASE-DISTILLATION PRETRAINING ON THE GNN
AGENT: BEST IN-TRAINING VALIDATION SCORE PER SEED, AND MEAN
ARGMAX TOTALS ON THE FULL HELD-OUT SPLITS (3 SEEDS FROM
SCRATCH, 2 PRETRAINED).

GNN variant	val-small best	Full val	Full test
From scratch	689 / 689 / 689	64 550	69 180
+ pretraining	651 / 668	57 980	63 229

totals by 9–10% on average, with the better seed within 0.5% of -Oz — the first deterministic policy in the study to beat the single-episode random null on both held-out splits. Pretraining repairs the policy’s *mode*; the from-scratch policy remains the better *sampler*. The two fixes are complementary: pretrain when only one rollout is affordable, sample when eight are.

D. Binary-level metrics

IC is a proxy, so we compiled the optimized modules to objects with the environment’s own LLVM 10 code generator and measured section sizes. On programs with at least 1000 instructions, relative IC reduction strongly predicts relative `.text` reduction (Pearson $r=0.78$, $p=2.2 \times 10^{-6}$); below that size, fixed code-generation overheads dominate and the proxy weakens — a boundary we report rather than hide. The IC gains translate into bytes: across the 347 paired battery programs, best-of-8 GNN sampling produces `.text` sections 10.6% smaller than -Oz (864 887 vs. 967 878 bytes; the random null achieves 10.4%), and across all 371 programs random search alone is 4.0% below -Oz. The discovered 45-pass sequences also apply *faster* than the -Oz pipeline (median 15.0–15.8 ms vs. 19.5 ms per module through the plain `opt` CLI, which also demonstrates the sequences work outside the RL environment). Total object footprint is dominated (>99.9%) by data and BSS sections, which are essentially invariant under pass ordering (<0.03% change), so `.text` is the informative size metric.

V. DISCUSSION AND LIMITATIONS

Three practices follow from these results. Evaluations of learned pass ordering should (1) report a random null inside the same action space and budget, since a curated space plus trivial search already beats -Oz; (2) report sampling-based in addition to deterministic evaluation, since the two can reverse rankings; and (3) never use -O3 as a size baseline. Our own limitations are measured rather than hypothetical: the study optimizes IR instruction count with a validated but size-bounded correlation to binary sections; training used four small programs by design (as the experimental control), and a mixed-size training arm did not improve generalization; runtime performance is not measured; and the stack is CompilerGym 0.2.5 (LLVM 10). The per-step cost of the GNN representation ($\sim 17\times$ the flat features during training) is real, though at deployment the cost of a sampled rollout is dominated by compilation itself.

VI. CONCLUSION

Under controlled measurement, the value of RL for LLVM pass ordering redistributes: most of the headline gain over $-\text{Oz}$ belongs to the curated action space; the genuine, statistically unambiguous contribution of the learned graph policy is that of an *amortized searcher* — greedy-search quality at a fraction of its compilation budget, transferable from four toy programs to seven foreign suites — and the encoder that carries it can only be trained with a pretraining step that costs an hour of CPU. Evaluation choices dominated representation choices at every step of this study; we suggest treating that as the default hypothesis in this domain.

REFERENCES

- [1] A. Haj-Ali, Q. Huang, W. Xiang, W. Moses, K. Asanovic, J. Wawrzyniek, and I. Stoica, “Autophase: Juggling HLS phase orderings in random forests with deep reinforcement learning,” in *Proceedings of Machine Learning and Systems (MLSys)*, 2020.
- [2] C. Cummins, B. Wasti, J. Guo, B. Cui, J. Ansel, S. Gomez, S. Jain, J. Liu, O. Teytaud, B. Steiner, Y. Tian, and H. Leather, “Compiler-Gym: Robust, performant compiler optimization environments for AI research,” in *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2022.
- [3] S. Jain, Y. Andaluri, S. VenkataKeerthy, and R. Upadrasta, “POSET-RL: Phase ordering for optimizing size and execution time using reinforcement learning,” in *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2022.
- [4] Y. Liang, K. Stone, A. Shamel, C. Cummins, M. Elhoushi, J. Guo, B. Steiner, X. Yang, P. Xie, H. Leather, and Y. Tian, “Learning compiler pass orders using coresets and normalized value prediction,” in *International Conference on Machine Learning (ICML)*, 2023.
- [5] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [6] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [7] C. Cummins, Z. Fisches, T. Ben-Nun, T. Hoefler, M. O’Boyle, and H. Leather, “ProGraML: A graph-based program representation for data flow analysis and compiler optimizations,” in *International Conference on Machine Learning (ICML)*, 2021.
- [8] M. Trofin, Y. Qian, E. Brevdo, Z. Lin, K. Choromanski, and D. Li, “MLGO: a machine learning guided compiler optimizations framework,” *arXiv preprint arXiv:2101.04808*, 2021.
- [9] R. Mammadli, A. Jannesari, and F. Wolf, “Static neural compiler optimization via deep reinforcement learning,” in *IEEE/ACM Workshop on the LLVM Compiler Infrastructure in HPC (LLVM-HPC)*, 2020.
- [10] C. Cummins, V. Seeker, D. Grubisic, B. Roziere, J. Gehring, G. Synnaeve, and H. Leather, “Meta large language model compiler: Foundation models of compiler optimization,” *arXiv preprint arXiv:2407.02524*, 2024.
- [11] M. R. Guthaus, J. S. Ringenberg, D. Ernst, T. M. Austin, T. Mudge, and R. B. Brown, “MiBench: A free, commercially representative embedded benchmark suite,” in *IEEE International Workshop on Workload Characterization (WWC)*, 2001.
- [12] Y. Hara, H. Tomiyama, S. Honda, H. Takada, and K. Ishii, “CHStone: A benchmark program suite for practical c-based high-level synthesis,” in *IEEE International Symposium on Circuits and Systems (ISCAS)*, 2008.
- [13] X. Yang, Y. Chen, E. Eide, and J. Regehr, “Finding and understanding bugs in C compilers,” in *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2011.